

Apache Kafka

Framework Unificado para Processamento Distribuído de Dados

Infraestrutura de Alta Vazão para ETL, Machine Learning e Análise de Dados em Tempo Real

80%

Fortune 500

50K+

Empresas

<5ms

Latência P99

2M+

msg/s

Stack Tecnológico

Infraestrutura de Ciência de Dados e Big Data

Escolha guiada por alta performance, baixa latência e ecossistema completo



Apache Spark

Processamento Distribuído

Framework unificado para processamento de dados em larga escala (batch e near real-time)

Casos: ETL, ML, análise de risco



Apache Flink

Processamento em Tempo Real

Framework streaming-first com baixa latência e estado consistente (stateful)

Casos: Detecção de fraudes, anomalias, risco



Apache Kafka

Mensageria/Ingestão

Plataforma de streaming de eventos distribuída, alta vazão e tolerante a falhas

Casos: Transações, logs, cotações



MinIO

Data Lake

Armazenamento S3-compatible para dados brutos e processados



PostgreSQL

Data Warehouse

Banco relacional robusto para BI e relatórios estruturados



Apache Airflow

Orquestração

Gestão de workflows complexos e pipelines de dados



Python + ML

Análise e ML

Pandas, Scikit-learn, TensorFlow, PyTorch

Sumário Executivo

01 Introdução ao Apache Kafka

Plataforma de streaming distribuída, características fundamentais e adoção empresarial

02 Arquitetura e Componentes

Producers, Brokers, Topics, Partitions, Consumers e KRaft Mode

03 Instalação e Configuração

Requisitos, KRaft Mode sem ZooKeeper, cluster multi-broker

04 Camada ETL

Kafka Connect, Kafka Streams, ksqlDB e pipelines de dados

05 Machine Learning e IA

Feature Stores, model serving, inferência em tempo real com Flink

06 Análise de Dados

Superset, Grafana, Jupyter Hub, Apache Pinot e Data Mining

07 Tecnologias Emergentes 2025-2026

Kafka 4.0, KRaft, Redpanda, Pulsar, Diskless Kafka, Iceberg, Agentic AI

CAPÍTULO 01

Introdução ao Apache Kafka

Plataforma de Streaming Distribuída de Código Aberto

Processamento em tempo real em escala empresarial

O que é Apache Kafka?

☰ Definição

Apache Kafka é uma plataforma de **streaming distribuída** de código aberto, desenvolvida pela Apache Software Foundation. Criada originalmente pelo **LinkedIn** e posteriormente doada para a Apache em **2011**.

Tornou-se o **padrão de fato** para processamento de streams de dados em tempo real em escala empresarial, processando trilhões de mensagens diariamente em produção.

📈 Adoção Global

80%

Fortune 500

50K+

Empresas

2M+

msg/segundo

<5ms

Latência P99

🎯 Objetivos da Arquitetura

- ✓ Demonstrar instalação completa do Apache Kafka em ambiente de produção
- ✓ Configurar cluster Kafka multi-broker com alta disponibilidade
- ✓ Integrar camadas de ETL, Machine Learning e Análise de Dados
- ✓ Apresentar tecnologias emergentes no ecossistema Kafka
- ✓ Fornecer referências e bibliotecas para implementação

📦 Escopo de um projeto

Abrange desde a **instalação básica** até a **configuração avançada** de um cluster Kafka, incluindo integração com ferramentas de processamento de streams, armazenamento de dados e frameworks de machine learning.

Características Fundamentais do Kafka



Alta Throughput

Capacidade de processar **milhões de mensagens por segundo** com baixa latência. Projetado para cargas de trabalho massivas em produção.

2M+

mensagens/segundo



Baixa Latência

Entrega de mensagens em **milissegundos**, ideal para aplicações em tempo real como detecção de fraude e monitoramento.

<5ms

latência P99



Escalabilidade Horizontal

Adicione **brokers dinamicamente** para aumentar capacidade sem downtime. Particionamento inteligente distribui carga.



escala horizontal



Tolerância a Falhas

Replicação de partições com líder e seguidores. Failover automático garante alta disponibilidade.

99.99%

disponibilidade



Persistência Durável

Mensagens persistidas em **disco com replicação**. Logs imutáveis permitem replay e reprocessamento.

Configurável

retenção de dados



Processamento em Tempo Real

Streams contínuos de dados processados instantaneamente. Integração nativa com ferramentas de stream processing.

Real-time

processamento

CAPÍTULO 02

Arquitetura e Componentes

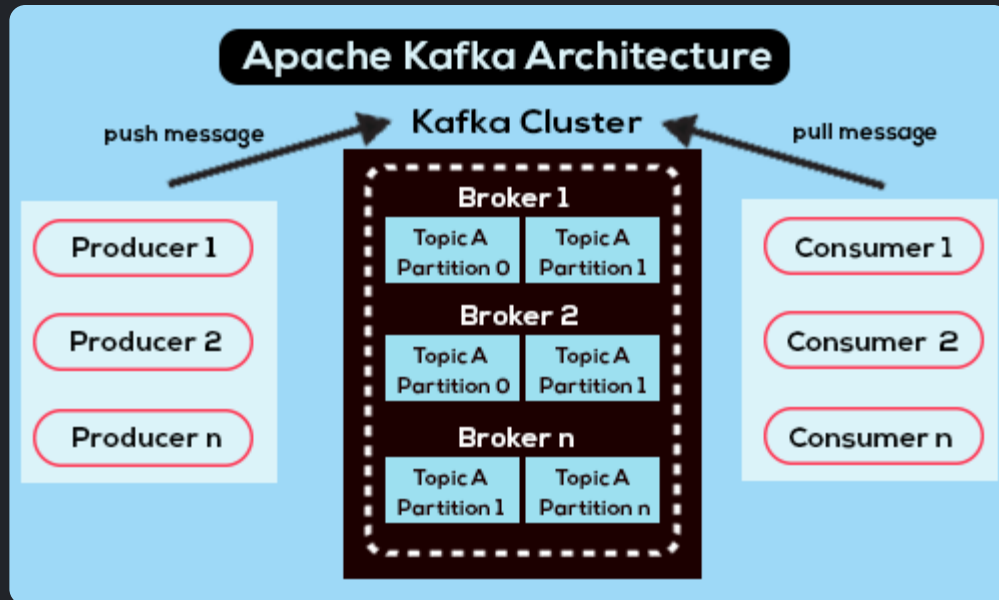
Estrutura Técnica do Sistema de Streaming

Modelo pub/sub distribuído de alta performance



Arquitetura Geral do Apache Kafka

Diagrama da Arquitetura Kafka



Fluxo de Dados

- 1 Producers Publicam**
Aplicações clientes enviam eventos para tópicos
- 2 Brokers Armazenam**
Servidores Kafka persistem dados em partições
- 3 Replicação Ocorre**
Partições replicadas entre brokers para HA
- 4 Consumers Processam**
Grupos de consumidores leem e processam dados

Modelo Pub/Sub Distribuído

Kafka opera em modelo **publicação-assinatura** onde:

- **Produtores** publicam em tópicos
- **Tópicos** categorizam mensagens
- **Consumidores** assinam tópicos

Componentes Principais em Detalhe

Producers (Produtores)

Aplicações clientes que **publicam eventos** em tópicos Kafka. Responsáveis por escolher a partição.

Features: Balanceamento automático, chaves para ordenação, confirmações configuráveis

Brokers

Servidores Kafka que **armazenam e servem dados**. Cluster multi-broker garante HA.

Identificação: Cada broker tem ID único, gerencia múltiplas partições

Topics (Tópicos)

Categorias ou feeds nomeados. Divididos em partições para paralelismo.

Particionamento: Cada tópico tem N partições distribuídas entre brokers

Consumers (Consumidores)

Lêem dados de tópicos. Organizados em **consumer groups** para processamento paralelo.

Balanceamento: Cada partição consumida por apenas um consumidor do grupo

KRaft Mode (Novo)

A partir da versão **3.0+**, Kafka introduziu modo KRaft que **elimina ZooKeeper**.

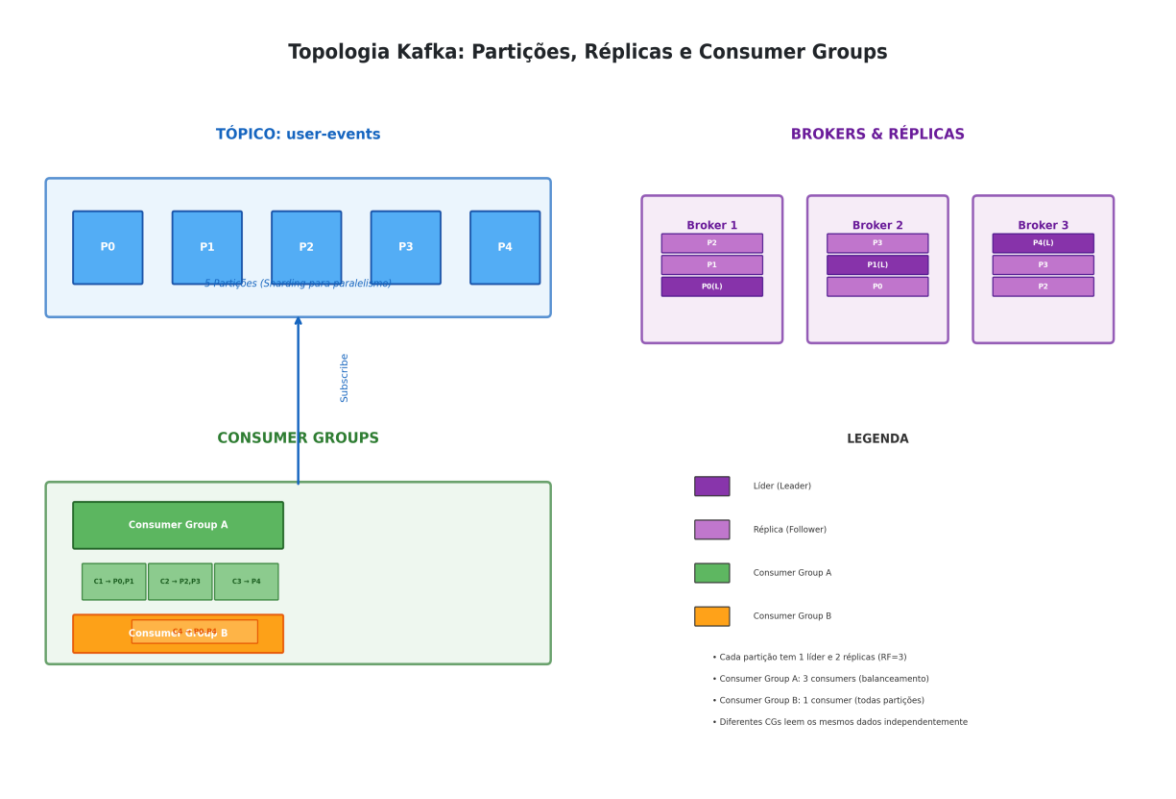
- ✓ Arquitetura simplificada
- ✓ Melhor escalabilidade
- ✓ Recuperação mais rápida
- ✓ Menor complexidade operacional

Kafka 4.0 - KRaft Only

Lançado em **março 2025**, Kafka 4.0 remove completamente o suporte a ZooKeeper. KRaft é o único protocolo de consenso.

Topologia e Particionamento

Topologia Kafka: Partições, Réplicas e Consumer Groups



Conceito de Particionamento

O **particionamento** é fundamental para escalabilidade horizontal. Cada tópico é dividido em partições configuráveis.

Partição = Log Ordenado e Imutável

Cada partição mantém sequência ordenada de mensagens com offset único

Paralelismo de Consumo

Número de partições determina grau máximo de paralelismo

Replicação de Partições

Garante **alta disponibilidade** através de réplicas distribuídas:

👑 Líder (Leader)

Processa todas as leituras e escritas

👤 Seguidores (Followers)

Replicam dados para failover

CAPÍTULO 03

Instalação e Configuração

Deploy de Cluster Kafka em Ambiente de Produção

KRaft Mode: ZooKeeper-free deployment



Requisitos e Preparação do Ambiente

Requisitos de Hardware

CPU

Mínimo: 4 cores | Recomendado: 8+ cores



Memória RAM

Mínimo: 8GB | Recomendado: 16-32GB



Disco (SSD)

SSD recomendado | Mínimo: 100GB livre



Rede

Conexão estável de 1Gbps ou superior



Requisitos de Software



Sistema Operacional

Linux (Ubuntu 20.04+, CentOS 7+, RHEL 8+)



Java Runtime

OpenJDK 11 ou superior (17+ recomendado)



Nota Importante

Kafka 4.0+ requer Java 17+ (Java 11 deprecated)



Checklist de Preparação

- ✓ Verificar recursos de hardware disponíveis
- ✓ Instalar Java 17+ e configurar JAVA_HOME
- ✓ Configurar hostname e resolução DNS
- ✓ Abrir portas necessárias no firewall
- ✓ Criar diretórios de dados com permissões

Projeto Pré-requisitos Estrutura de Instalação e Configuração - KRaft Mode

Download e Extração

```
# Download do Kafka 3.6.1+
wget https://downloads.apache.org/kafka/3.6.1/kafka_2.13-3.6.1.tgz
# Extrair arquivo
tar -xzf kafka_2.13-3.6.1.tgz
cd kafka_2.13-3.6.1
```

Configuração KRaft

```
# Gerar ID do cluster
KAFKA_CLUSTER_ID="$(bin/kafka-storage.sh random-uuid)"
# Formatar storage
bin/kafka-storage.sh format \
  -t $KAFKA_CLUSTER_ID \
  -c config/kraft/server.properties
```

Iniciar o Servidor

```
# Iniciar broker Kafka
bin/kafka-server-start.sh \
  config/kraft/server.properties
```

Configuração do Servidor

```
# Editar config/kraft/server.properties# Identificação do nó
node.id=1
# Quorum de controllers
controller.quorum.voters=1@localhost:9093
# Listeners
listeners=PLAINTEXT://:9092,CONTROLLER://:9093
# Diretório de logs
log.dirs=/var/lib/kafka/logs
```

★ Vantagens do KRaft

- ✓ **Arquitetura Simplificada:** Sem ZooKeeper separado
- ✓ **Melhor Escalabilidade:** Metadados mais eficientes
- ✓ **Recuperação Rápida:** Startup mais rápido
- ✓ **Menor Complexidade:** Menos componentes

Estrutura de Configuração de Cluster Multi-Broker

⚙️ Configuração de 3 Brokers

```
# Configuração para cluster com 3 brokers# Cada broker precisa de node.id único# Broker 1 - config/kraft/server-1.properties
node.id=1
listeners=PLAINTEXT://:9092,CONTROLLER://:9093
log.dirs=/var/lib/kafka/logs-1
# Broker 2 - config/kraft/server-2.properties
node.id=2
listeners=PLAINTEXT://:9094,CONTROLLER://:9095
log.dirs=/var/lib/kafka/logs-2
# Broker 3 - config/kraft/server-3.properties
node.id=3
listeners=PLAINTEXT://:9096,CONTROLLER://:9097
log.dirs=/var/lib/kafka/logs-3
# Quorum compartilhado (mesmo em todos)
controller.quorum.voters=\
  1@localhost:9093,2@localhost:9095,3@localhost:9097
```

🔧 Testando a Instalação

```
# Criar um tópico de teste
bin/kafka-topics.sh --create --topic test-topic \
  --bootstrap-server localhost:9092 \
  --partitions 3 --replication-factor 3
# Listar tópicos
bin/kafka-topics.sh --list \
  --bootstrap-server localhost:9092
# Verificar partições e réplicas
bin/kafka-topics.sh --describe --topic test-topic \
  --bootstrap-server localhost:9092
```

Arquitetura de Cluster

Alta Disponibilidade

Múltiplos brokers garantem continuidade em caso de falha

Balanceamento de Carga

Partições distribuídas entre brokers

Escalabilidade Horizontal

Adicione brokers conforme necessidade

💡 Melhores Práticas

- ✓ Use número ímpar de controllers (3, 5, 7)
- ✓ Mínimo 3 controllers para produção
- ✓ SSD para controllers (metadados rápidos)
- ✓ Rede dedicada para comunicação interna
- ✓ Monitore métricas de quorum Raft

CAPÍTULO 04

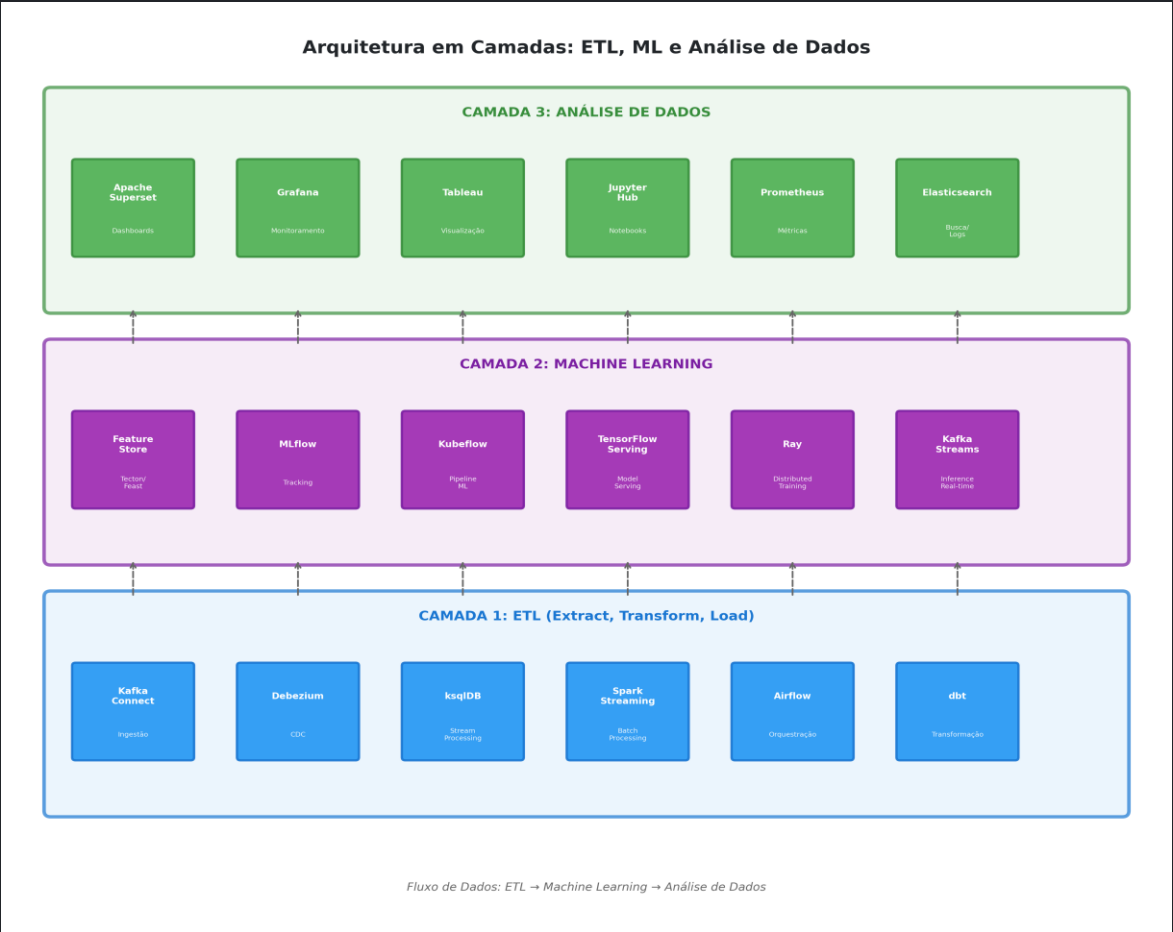
Camada ETL

Extract, Transform, Load em Tempo Real

Pipelines de dados contínuos e confiáveis

Pipeline ETL com Kafka

Arquitetura em Camadas - ETL, ML e Análise



Camada ETL - Responsabilidades

A camada **ETL (Extract, Transform, Load)** é responsável por todo o ciclo de vida dos dados:

- E Extract (Extração)**
Ingestão de dados de múltiplas fontes: BD, APIs, logs, IoT
- T Transform (Transformação)**
Limpeza, enriquecimento, agregação, normalização
- L Load (Carga)**
Escrita em destinos: data lakes, warehouses, APIs

Ferramentas do Ecossistema



Kafka Connect
Integração



Kafka Streams
Processamento



ksqlDB
SQL Streams

Kafka Connect - Integração de Dados



O que é Kafka Connect?

Kafka Connect é um framework para conectar Kafka com sistemas externos de forma **escalável e confiável**. Elimina necessidade de código customizado.

→ Source Connectors

Importam dados **para** Kafka (BD, APIs, arquivos)

← Sink Connectors

Exportam dados **de** Kafka (data lakes, índices)

Benefícios

- ✓ **Escalabilidade:** Paralelismo automático
- ✓ **Tolerância a Falhas:** Recuperação automática
- ✓ **Gerenciamento Centralizado:** API REST
- ✓ **Ecosystem:** 100+ conectores disponíveis

</>Exemplo: JDBC Source Connector

```
# Criar conector via API REST
curl -X POST http://localhost:8083/connectors \
-H "Content-Type: application/json" \
-d '{
  "name": "jdbc-source-connector",
  "config": {
    "connector.class": \
      "io.confluent.connect.jdbc.JdbcSourceConnector",
    "connection.url": \
      "jdbc:postgresql://localhost:5432/mydb",
    "connection.user": "admin",
    "connection.password": "secret",
    "mode": "incrementing",
    "incrementing.column.name": "id",
    "topic.prefix": "db-",
    "poll.interval.ms": "5000"
  }
}
```

Conectores Populares

Source

JDBC, Debezium CDC, S3, Elasticsearch

Sink

S3, HDFS, BigQuery, Snowflake, MongoDB

Kafka Streams e ksqlDB

Kafka Streams

Biblioteca cliente para construir aplicações e microsserviços de processamento de streams.

Características

- Java/Scala API completa
- Controle programático total
- Stateful processing
- Embedded em aplicações

Quando Usar

- ✓ Lógica complexa customizada
- ✓ Integração com código existente
- ✓ Controle fino de performance

ksqlDB

Motor SQL para processamento de streams. Processa dados em tempo real usando SQL.

Características

- SQL familiar
- Sem código necessário
- Materialized views
- Pull queries (0.29+)

Quando Usar

- ✓ Desenvolvimento rápido
- ✓ Times com expertise SQL
- ✓ Prototipagem ágil

Exemplo ksqlDB

```
-- Criar stream a partir de tópicoCREATE STREAM user_events (  
  user_id VARCHAR,  
  event_type VARCHAR,  
  timestamp BIGINT  
) WITH (  
  KAFKA_TOPIC='user-events',  
  VALUE_FORMAT='JSON'  
);  
-- Agregação em tempo realCREATE TABLE event_counts AS  
SELECT event_type, COUNT(*) AS total  
FROM user_events  
WINDOW TUMBLING (SIZE 1 MINUTE)  
GROUP BY event_type;
```

CAPÍTULO 05

Machine Learning e IA

Pipelines de ML em Tempo Real

Inferência contínua e model serving

Integração Kafka + Machine Learning



Pipeline ML End-to-End

Integração de Kafka com frameworks de ML permite construir **pipelines em tempo real**: desde ingestão de dados de treinamento até deploy.

1. Ingestão de Dados

Eventos de treinamento e features em tempo real via Kafka

2. Feature Engineering

Transformações com Kafka Streams/ksqlDB

3. Treinamento

Modelos treinados com dados do Kafka

4. Deploy e Inferência

Modelos servidos para predição em tempo real

Componentes do Ecossistema



Feature Store

Feast, Tecton - armazenam e servem features

Integração direta com Kafka para features em tempo real



MLflow

Plataforma open source para ciclo de vida de ML

Experimentação, reprodução, deploy, registro de modelos



TensorFlow Serving

Sistema de serving flexível e alta performance

Integração via Kafka para inferência em tempo real

Casos de Uso

- ✓ Detecção de fraude em tempo real
- ✓ Recomendações personalizadas
- ✓ Anomalias em IoT
- ✓ Predição de churn

Padrões de Model Serving

1 Embedded Models

Modelos executam **dentro** de aplicações de stream processing (Flink, Kafka Streams).

Prós

- Sem latência de rede
- Batch predictions
- Controle total

Contras

- Acoplamento forte
- Difícil atualizar

<5ms

latência P99

2 Sidecar Pattern

Modelos deployados como **containers sidecar** junto aos pods da aplicação.

Prós

- Independência de linguagem
- Escalabilidade separada
- Isolamento de recursos

Contras

- Overhead de comunicação
- Contenção de recursos

5-20ms

latência P99

3 Dedicated Servers

Infraestrutura centralizada: **KServe, Ray Serve, BentoML**.

Prós

- Escalabilidade independente
- Hardware especializado (GPUs)
- A/B testing, versionamento

Contras

- Latência de rede
- Infraestrutura adicional

20-100ms

latência P99

Comparação de Performance

Embedded

Throughput: **100K+ req/s**

Sidecar

Throughput: **50K req/s**

Dedicated

Throughput: **10K+ req/s**

Inferência em Tempo Real com Flink

Pipeline de Inferência

- 1

Ingestão de Dados
Kafka recebe eventos de múltiplas fontes (IoT, web apps, transações)
- 2

Pré-processamento
Flink: limpeza, transformação, feature engineering
- 3

Inferência do Modelo
Chamadas ao model server (S3, TensorFlow, custom)
- 4

Pós-processamento
Filtragem, agregação, enriquecimento com contexto
- 5

Saída
Dashboards, bancos de dados, ações automatizadas

Exemplo: ML_PREDICT no Flink

```
-- Criar conexão com Azure OpenAICREATE MODEL embedding_model
INPUT (text STRING)
OUTPUT (response ARRAY<FLOAT>)
WITH (
  'provider' = 'azureopenai',
  'task' = 'embedding'
);
-- Inferência em tempo realSELECT
id,
text,
embedding
FROM product_updates,
LATERAL TABLE(
  ML_PREDICT('embedding_model', text)
);
```

Performance em Produção



CAPÍTULO 06

Análise de Dados e Data Mining

Visualização e Mineração de Dados em Streaming

Dashboards em tempo real e analytics



Ferramentas de Análise e Visualização

Apache Superset

Aplicação web moderna para **exploração e visualização** de dados. Interface intuitiva.

Features

- Dashboards interativos
- SQL Lab para queries
- 40+ tipos de visualização
- Cache de resultados

Integração com PostgreSQL, MySQL, Druid, Presto

Grafana

Plataforma de **analytics e monitoramento** interativa. Plugin Kafka para dashboards.

Features

- Dashboards em tempo real
- Alertas configuráveis
- 100+ data sources
- Visualizações ricas

Monitoramento de tópicos, lag de consumidores, métricas

Jupyter Hub

Notebooks **multi-usuário** para análise exploratória de dados.

Features

- Python, Scala, R kernels
- Colaboração em tempo real
- Integração com Spark
- Visualizações interativas

Exploração de dados Kafka com kafka-python

Fluxo de Dados em Tempo Real



Kafka Topics
Eventos brutos



Processamento
Streams/Flink



Data Store
Pinot/Druid



Dashboards
Superset/Grafana

Data Mining e Apache Pinot

Apache Pinot

Banco de dados analítico para streaming, projetado para ingestão e query de dados com latência sub-segundo.

Ingestão Contínua

Streaming direto de Kafka, Pulsar, Kinesis sem delays de batch

Query Instantânea

Dados disponíveis para query em milissegundos após ingestão

Dados Híbridos

Combinação de streams em tempo real com dados históricos

Diferença para Bancos Tradicionais

Snowflake/Redshift

Micro-batch

Latência: minutos

Apache Pinot

Streaming

Latência: milissegundos

Performance Extrema

30K+

Queries/segundo

<1s

Latência P99

PBs

Dados históricos

Real-time

Ingestão

Casos de Uso

- ✓ Dashboards em tempo real para BI
- ✓ AI-driven automation e anomaly detection
- ✓ Customer-facing analytics (user-facing)
- ✓ Monitoramento de métricas de negócio

Indexação durante Ingestão

Diferente de bancos tradicionais que indexam **após** ingestão, o Pinot indexa **durante** a ingestão, eliminando período onde registros não são pesquisáveis.

CAPÍTULO 07

Tecnologias Emergentes 2025-2026

Inovações no Ecossistema de Streaming

Kafka 4.0, KRaft, Redpanda, Iceberg e mais

Apache Kafka 4.0 e KRaft Mode



Kafka 4.0 Lançado!

Março 2025

A versão **4.0** representa uma mudança arquitetural monumental: **remoção completa do ZooKeeper**.

Mudanças Principais

✗ ZooKeeper Removido

KRaft é o único protocolo de gerenciamento de metadados

☑ MirrorMaker 1 Removido

Use MirrorMaker 2.0 para replicação

☑ Log4j2 Completo

Transição completa de Log4j para Log4j2

⚠ Java 11 Deprecated

Requer Java 17+ para Kafka 4.0

Vantagens do KRaft



Arquitetura Simplificada

Sem necessidade de gerenciar cluster ZooKeeper separado



Melhor Escalabilidade

Handling de metadados mais eficiente para clusters grandes



Recuperação Mais Rápida

Startup e failover mais rápidos



Dynamic KRaft Quorums

Adicione/remova controllers sem downtime (KIP-853)

Requisitos de Migração

- Kafka 3.3.x-3.9.x KRaft: upgrade direto para 4.0
- Kafka 3.3.x-3.9.x ZooKeeper: migre para KRaft primeiro
- Kafka <3.3.x: upgrade para 3.9.x, depois migre

Alternativas Modernas: Redpanda e Pulsar

Redpanda

Alternativa moderna compatível com API Kafka, escrita em C++ com framework Seastar.

Vantagens

- 10x latências mais baixas (P99.9)
- 3-6x mais eficiente em custo
- Sem ZooKeeper ou JVM
- Single binary, fácil deploy
- Thread-per-core architecture

Performance

122K
msg/s

5.4ms
latência

Throughput 44.7% melhor que Kafka

Apache Pulsar

Plataforma de streaming com arquitetura em camadas e recursos avançados.

Diferenciais

- Tiered Storage nativo
- Multi-tenancy nativo
- Geo-replicação integrada
- Arquitetura em camadas (BookKeeper)
- Unified messaging + streaming

Performance

1.8M
msg/s

20-35ms
latência P99

Ideal para multi-tenant e geo-distribuído

Benchmark Comparativo

Kafka

Throughput: 67K msg/s
Latência: 5.8ms

Redpanda

Throughput: 122K msg/s
Latência: 5.4ms

Pulsar

Throughput: 1.8M msg/s
Latência: 20-35ms

RabbitMQ

Throughput: 54K msg/s
Latência: 146ms

Tendências Emergentes 2025-2026

1 Consolidação do Ecosistema

Provedores consolidados ganham terreno. **Market consolidation** com vendedores saindo ou pivotando.

Forrester Wave 2025 destaca evolução para plataformas completas

2 Diskless Kafka + Iceberg

Tiered Storage e **Apache Iceberg** criam nova fundação de armazenamento.

Dados antigos automaticamente movidos para S3/GCS

3 Real-time Analytics

Analytics em tempo real torna-se parte da **camada de streaming**, não downstream.

Eliminação de delays entre ingestão e análise

4 SLAs Zero Data Loss

Empresas demandam **SLAs com zero perda de dados** e failover seamless.

Multi-AZ deployment com state preservation

5 Regional Cloud

Deployments regionais impulsionados por compliance e soberania de dados.

GDPR, LGPD e regulamentações locais

6 Agentic AI

Streaming alimenta **Agentic AI** com contexto e inferência de modelos em tempo real.

LLMs + streaming para decisões autônomas



Insight: O ecossistema de streaming está evoluindo de **infraestrutura de mensagens** para **plataforma completa** com governança, observabilidade e suporte a IA integrados.

Novas Tecnologias de Processamento

Materialize

Motor de processamento de streams **SQL** que mantém views materializadas atualizadas em tempo real.

Use case: Dashboards sempre atualizados sem polling

Apache Iceberg

Formato de **tabela open source** para grandes conjuntos de dados analíticos. Integração com Kafka para lakehouses.

Use case: Lakehouse architecture com dados Kafka

RisingWave

Banco de dados de **streaming distribuído** para consultas SQL contínuas em dados em movimento.

Use case: Análise contínua de streams complexos

Cruise Control

Sistema da **LinkedIn** para automatizar operações de clusters Kafka: balanceamento, anomalias, autorreparação.

Features: Auto-rebalancing, anomaly detection

Tiered Storage

Dados antigos **movidos automaticamente** para armazenamento de objeto (S3, GCS), reduzindo custos.

Benefício: 50-70% redução de custos de storage

OpenTelemetry

Instrumentação padronizada para **tracing distribuído** e métricas. Essencial para observabilidade.

Integração: Métricas Kafka + traces distribuídos

Conclusão da aula e Próximos Passos

Professor Benir Falcão Pistille

✓ Conclusões

- Kafka evolui como componente central em arquiteturas modernas
- Kafka 4.0 + KRaft simplificam e melhoram a plataforma
- Integração ML/IA abre novas possibilidades
- Ecossistema rico com ferramentas especializadas

☰ Recomendações

- Teste em ambiente não-produtivo antes de migrar
- Considere Redpanda para casos de performance crítica
- Invista em observabilidade e automação
- Avalie Tiered Storage para redução de custos

Apache Kafka

Framework Unificado para Processamento Distribuído



user@kafka.apache.org



kafka.apache.org



github.com/apache/kafka